NimbusPay ticket triage - measured results

Base: Qwen/Qwen2.5-0.5B-Instruct (494M parameters). Best: clean_qlora; 4-bit load: **True**. Dev exact match **58.5%**, mean field accuracy **92.6%**.

Audit and choices

Only train.jsonl and SCHEMA.md informed cleaning. 1445 rows → 1350; 95 dropped, 175 fixed. Findings: priority_business_rule=60, migrate_legacy_prio_to_priority=70, migrate_legacy_date_to_ISO=30, placeholder_ticket=10, invalid_json=25, normalize_transaction_id=20, amount_conversion=25, duplicate=60. Counts of repairs overlap. Exact system/user duplicates retain the first occurrence. Invalid/truncated JSON and placeholder inputs are dropped rather than reconstructed. Legacy priority/date fields and IDs are normalized. Explicit claim amounts use Decimal arithmetic; balances and limits are excluded. Priority is recalculated from the ordered thresholds and legal threats. Categories/language are retained rather than relabelled with an AI model. Quoted threads are kept in training inputs to teach distraction handling. cleaning_log.csv records every changed row; audit.py provides executable evidence.

Training and controlled comparisons

Hugging Face Transformers + PEFT + bitsandbytes keeps the training loop inspectable and saves standard PEFT adapters. Raw training token lengths: median=168, p95=225, max=2215; length=2304, with no training truncation. The native Qwen chat template supports system messages. Prompt and padding tokens are masked; assistant JSON and end-of-turn tokens receive loss. No packing. All runs use seed 2026, batch 2 × accumulation 8, AdamW, LR 2e-4, cosine schedule, 5% warmup, dropout .05, and projection-module LoRA (alpha/r=2). The full clean_qlora candidate uses all 1,350 cleaned rows and two epochs. The four controlled comparisons use one-epoch pilots: 500 raw rows sampled with seed 2026 and their retained cleaned counterparts. Every pilot is compared only to pilot_clean; differences from the full-data candidate are not controlled evidence. This limited cohort reduces power and may underrepresent rare errors; it was chosen after measured GPU speed made five full runs impractical before the deadline. best trainable parameters=8,798,208. Dev is evaluation-only. Selecting the winner on dev can overfit dev; hidden test is never used for selection.

Run (rows; epochs)	Change relative to pilot_clean	Dev exact	Mean fields	Peak GPU GiB	Train min
clean_qlora (1350; 2)	full-data candidate; separate scope	58.5%	92.6%	9.03	14.63
pilot_clean (472; 1)	pilot reference	13.5%	79.2%	9.80	2.58
pilot_raw (500; 1)	raw data only	26.5%	80.1%	9.71	2.87
pilot_lora (472; 1)	16-bit weights only	19.5%	80.0%	9.51	1.90
pilot_rank8 (472; 1)	rank 8 only; alpha/r fixed	6.5%	74.8%	9.69	2.77
pilot_full_loss (472; 1)	all-token loss only	5.5%	75.4%	9.71	2.67

The raw-data pilot beat the cleaned pilot by 13.0 percentage points exact match; 16-bit LoRA gained 6.0 points. Rank 8 lost 7.0 points and all-token loss lost 8.0 points. These are single-seed pilot results, not evidence that cleaning hurts at full scale. Dropping rows also reduces optimizer steps at a fixed epoch count (500 raw vs 472 clean); a fixed-update follow-up would test that exposure effect. 16-bit LoRA was faster with slightly lower peak memory here, so 4-bit loading did not reduce measured training peak for this small model and protocol.

Both base-model baselines use 16-bit base weights and are measured with score.py:

Prompt	Dev exact	Mean fields	JSON valid
fixed	0.0%	0.0%	1.0%
rules	0.0%	2.1%	6.0%

Error analysis

Weakest fields: priority 72.0%, needs_human 82.0%, txn_date 93.0%. Weakest categories: refund 40.0% (n=55), payment_failed 41.5% (n=53), fraud 60.0% (n=15). Explanations below identify the failed rule; they are diagnostic hypotheses, not causal proof. Full tickets, labels and raw outputs are in dev_failures.json.

Dev ID	Specific mismatch (gold → prediction)	Explanation
dv-00005	priority: "P2" → "P3"	The ₹49,680.00 payment/refund requires P2 under the ₹5,000 threshold rule.
dv-00006	txn_date: "2026-06-30" → "2026-07-01"	"Day before yesterday" from 2026-07-02 requires 2026-06-30, not yesterday.
dv-00011	priority: "P1" → "P3"; needs_human: true → false	The ₹84,137.00 claim exceeds the ₹50,000 P1 threshold; P1 also requires escalation.
dv-00019	category: "refund" → "product_accessibility"; priority: "P1" → "P2"; needs_human: true → false	Generated an out-of-schema category for a ticket labelled refund.
dv-00037	priority: "P1" → "P3"; amount_paise: 28466900 → 1800000; txn_date: "2026-09-06" → "2028-09-06"; needs_human: true → false	The stated ₹284,669 became ₹18,000 and an omitted year became future 2028; the correct high-value claim requires P1 and escalation.
dv-00181	channel: "card" → null	The bank-card/transaction wording is labelled card; the model omitted that explicit mode.
dv-00012	priority: "P1" → "P3"; needs_human: true → false	The ₹237,415.00 claim exceeds the ₹50,000 P1 threshold; P1 also requires escalation.
dv-00013	priority: "P3" → "P2"	The ₹55.00 claim is below ₹5,000; urgency wording and account balance do not trigger P2.
dv-00014	needs_human: true → false	The ticket says "again and again", indicating repeated support contact despite non-P1 priority.
dv-00016	priority: "P2" → "P3"	The ₹25,647.00 payment/refund requires P2 under the ₹5,000 threshold rule.

Next week, reproduction and AI assistance

Next: inspect the 10 failures, construct train-only boundary/format examples from SCHEMA.md, test a 1.5B model, and run three seeds with confidence intervals. Hold out a training-only validation split before further dev selection. Check calendar arithmetic and adversarial multi-transaction wording separately. More rank alone is unlikely to solve systematic extraction errors. Inference is native chat formatting followed by plain greedy model.generate(max_new_tokens=200). Decode excludes control tokens; whitespace stripping is the only output transformation. No rules/examples, constrained decoding, field filling, external model or API is used to produce labels/predictions. The final saved adapter is reloaded before test generation and its first eight outputs are checked for reproducibility. OpenAI Codex assisted with audit code, training/notebook implementation, local verification, Colab operation and this report generator. All scores, memory and timing figures come from the executed pipeline. Training-label repairs use deterministic code and the provided schema; Codex was not used as a labelling or prediction model. The candidate must review and be able to explain each line.

Final adapter tensors are stored in fp16. Final dev/test predictions were regenerated from those saved tensors; the reported experiment scores retain the original training measurements.